
Referencias open-source para el desarrollo del CubeSat

*20 repositorios y recursos curados
para construir flight software, ground station y simulacion*

Proyecto: CubeSat-EdgeAI-Payload
Plataforma: INTISAT — payload de microscopia FPM
Audiencia: Equipo tecnico de 4 personas
Plan asociado: 2 semanas de exploracion en paralelo
Repositorio: github.com/JaminYC/CubeSat-EdgeAI-Payload
Fecha: 8 de mayo de 2026
Documento: v1.0

Resumen. Compilacion curada de 20 repositorios open-source y recursos academicos relevantes para el desarrollo de un CubeSat con IA a bordo. Cubre flight software (NASA cFS, F'), ground station (OpenMCT), simulacion (42, GMAT), autonomia (PLEXIL), CubeSats academicos completos (OreSat, UPSat, AAUSAT) e IA en orbita (ESA Phi-Lab). Para cada referencia incluye URL, descripcion, prioridad para el proyecto, y como aplicarla concretamente al payload INTISAT. Acompaña al plan de exploracion paralela del equipo de 4 personas durante 2 semanas.

Índice general

1. Como leer este documento	2
2. NASA — el clasico (4 referencias)	3
2.1. 1. NASA cFS — Core Flight System	3
2.2. 2. NASA F'(FPrime)	3
2.3. 3. NASA OpenMCT	4
2.4. 4. NASA Astrobee	4
3. NASA — apps puntuales de cFS (3 referencias)	5
3.1. 5. cFS sample_app	5
3.2. 6. cFS Limit Checker (LC)	5
3.3. 7. cFS Scheduler (SCH)	5
4. Simulacion y planeamiento (3 referencias)	7
4.1. 8. NASA 42 simulator	7
4.2. 9. NASA GMAT	7
4.3. 10. NASA Trick	7
5. Autonomia y vision (2 referencias)	8
5.1. 11. NASA PLEXIL	8
5.2. 12. JPL Open Source Rover	8
6. ESA y la IA en orbita (3 referencias)	9
6.1. 13. ESA Phi-Lab	9
6.2. 14. ESA Operations	9
6.3. 15. ESA OPS-SAT experiments	10
7. CubeSats academicos open-source (3 referencias)	11
7.1. 16. OreSat (Portland State University)	11
7.2. 17. UPSat (Libre Space Foundation)	11
7.3. 18. AAUSAT-3 / -4 (Aalborg University)	12
8. Estandares y handbooks (2 referencias)	13
8.1. 19. CalPoly CubeSat Design Specification	13
8.2. 20. NASA SE Handbook (Systems Engineering)	13
9. Plan de exploracion para el equipo de 4 personas	14
9.1. Cronograma	14
9.2. Output del equipo al final de las 2 semanas	14
10.Lectura priorizada (top 5 si solo tenes tiempo limitado)	15
11.Recursos extra que no son repositorios pero suman	16
11.1. NASA Software Catalog	16
11.2. Small Satellite Conference USU — Open Access Papers	16
11.3. ESA SAVOIR architecture	16
11.4. NASA cFS resource center	16
12.Cierre	17

1. Como leer este documento

Las 20 referencias estan ordenadas por categoria y dentro de cada categoria por **prioridad para tu proyecto**. Cada entrada incluye:

- **Nombre y URL** del repositorio o recurso.
- **Tipo:** framework / app / libro / paper / dashboard / etc.
- **Que es:** descripcion corta.
- **Como aplicalo al INTISAT:** caja azul.
- **Prioridad:** ★★★ (top), ★★ (medio), ★ (futuro / nice-to-have).
- **Tiempo estimado de exploracion.**

Recomendacion

Si el equipo es de 4 personas y solo tienen 2 semanas, atacar primero las marcadas con ★★★. Cada miembro toma una referencia distinta y la presenta al equipo al final.

2. NASA — el clasico (4 referencias)

2.1. 1. NASA cFS — Core Flight System

- URL: <https://github.com/nasa/cFS>
- Tipo: **Framework de flight software**
- Prioridad: ★★ (medio — arquitectura util)
- Tiempo: 5–10 dias

Que es: framework de NASA para flight software. Goddard lo usa desde 2007 en TODAS las misiones. Diseñado para RTOS pero corre tambien en Linux.

Componentes clave:

- cFE (Core Flight Executive): runtime con publish/subscribe.
- OSAL: capa de abstraccion sobre Linux/RTEMS/FreeRTOS/VxWorks.
- PSP: adaptacion a hardware.
- Apps: limit checker (LC), memory manager (MM), file manager (FM), data storage (DS), housekeeping (HK), scheduler (SCH).

Como usarlo en tu proyecto

El **patron arquitectural** de cFS (publish/subscribe + tablas de configuracion + FDIR) es directamente aplicable a tu daemon. NO copiar codigo (es complejo), sino imitar la estructura: cada subsistema publica mensajes, otros se suscriben, hay tablas de configuracion en tiempo de ejecucion. Tu `cubesat/daemon.py` ya hace algo similar via `/run/cubesat/`.

2.2. 2. NASA F'(FPrime)

- URL: <https://github.com/nasa/fprime>
- Tipo: **Framework de flight software (moderno)**
- Prioridad: ★★★ (alto)
- Tiempo: 3–7 dias

Que es: framework alternativo a cFS, mas reciente, mas simple, mas accesible. Usado en misiones reales: **MarCO** (los CubeSats interplanetarios que escoltaron a InSight) e **Ingenuity** (el helicoptero de Mars).

Por que vale:

- Mucho mejor documentado que cFS.
- Tutorial oficial paso a paso: <https://nasa.github.io/fprime/Tutorials/index.html>
- Generador de codigo a partir de XML.
- Hello World corre en 1 hora.

Como usarlo en tu proyecto

La opcion mas viable si quieren **adoptar un framework formal** en el INTISAT. Puede coexistir con tu daemon Python: F'maneja la coordinacion inter-subsistema y tu daemon corre el pipeline IA como un componente F'.

2.3. 3. NASA OpenMCT

- URL: <https://github.com/nasa/openmct>
- Tipo: **Dashboard de mission control**
- Prioridad: ★★★ (alto)
- Tiempo: 2 dias

Que es: framework web (JavaScript) para construir interfaces de control de misiones. Lo usa JPL en operaciones reales. Permite integrar telemetria de cualquier fuente y armar sinopticos, graficos, alarmas.

Por que vale:

- Te ahorra meses de desarrollo de tu ground station.
- Tutorial en 30 minutos te deja un dashboard andando.
- Plugin architecture: conectas tu fuente de telemetria.
- Imprimible, exportable, theme-able.

Como usarlo en tu proyecto

Una persona del equipo deberia probarlo esta semana. Conectar un script Python que lea `/run/cubesat/telemetry.json` (lo que ya publica tu daemon) y servirlo a OpenMCT por WebSocket. En 2 horas tienen el primer prototipo del dashboard del INTISAT.

2.4. 4. NASA Astrobee

- URL: <https://github.com/nasa/astrobee>
- Tipo: **Software completo de un robot espacial**
- Prioridad: ★★ (medio)
- Tiempo: 3 dias para entender, semanas para profundizar

Que es: el flight software de los 3 robots free-flying que vuelan dentro de la ISS. Open source completo: vision, navegacion, planificacion, comunicacion, control.

Como usarlo en tu proyecto

Util si quieren ver como NASA estructura un sistema con **vision computacional + autonomia + telemetria** todo junto. Tu payload tiene la misma combinacion (vision + IA + telemetria) en menor escala.

3. NASA — apps puntuales de cFS (3 referencias)

3.1. 5. cFS sample_app

- URL: https://github.com/nasa/sample_app
- Tipo: App de ejemplo de cFS
- Prioridad: ★★★ (alto si quieren entender cFS)
- Tiempo: 4–8 horas

Que es: el "Hello World" de cFS. Una app minima que muestra como se estructura cualquier subsistema en el ecosistema cFS. Ideal para aprender los conceptos sin perderse en el monstruo completo.

Como usarlo en tu proyecto

Antes de leer cFS completo, leer sample_app. Te explica: publish/subscribe, telemetria, comandos, tablas de configuracion. En 4 horas ya tenes los conceptos.

3.2. 6. cFS Limit Checker (LC)

- URL: <https://github.com/nasa/lc>
- Tipo: App de FDIR
- Prioridad: ★★
- Tiempo: 1 dia

Que es: monitorea telemetria contra limites configurables y dispara acciones de FDIR (Fault Detection, Isolation, Recovery) cuando algo sale de rango. Ej: si la temperatura de la CPU supera 80°C, dispara un comando de SAFE_MODE.

Como usarlo en tu proyecto

Tu daemon hoy NO tiene FDIR formal. Esta app es la **plantilla de referencia** para implementarlo. Conceptos: tabla de limites, watchpoints, actionpoints, response actions.

3.3. 7. cFS Scheduler (SCH)

- URL: <https://github.com/nasa/sch>
- Tipo: Scheduler de mision
- Prioridad: ★ (futuro)
- Tiempo: 1 dia

Que es: ejecuta tareas periodicas (housekeeping, telemetria, recolecciones cientificas) segun un schedule configurable. Ej: cada 30 s publica HK, cada 1 hora hace un scan de payload, etc.

Como usarlo en tu proyecto

Tu sistema es REACTIVO (espera comandos del OBC), no SCHEDULED. Pero si en algun momento quieres un modo "autonomo cientifico" donde la payload decida sola cuando capturar, el patron de SCH es el de referencia.

4. Simulacion y planeamiento (3 referencias)

4.1. 8. NASA 42 simulator

- URL: <https://github.com/nasa/42>
- Tipo: **Simulador 6-DOF de spacecraft**
- Prioridad: ★★ (alto si tienen ADCS team)
- Tiempo: 2–3 dias para arrancar

Que es: simulador completo de dinamica de naves espaciales. Calcula orbita, actitud, sensores, actuadores, perturbaciones. Permite probar algoritmos ADCS sin satellite real.

Como usarlo en tu proyecto

El equipo ADCS del INTISAT (no vos, el de actitud) puede probar sus algoritmos aqui antes del integration test. Para tu payload no aplica directamente.

4.2. 9. NASA GMAT

- URL: <https://github.com/nasa/GMAT>
- Tipo: **General Mission Analysis Tool**
- Prioridad: ★★ (alto para planeamiento)
- Tiempo: 1 dia para basicos

Que es: la herramienta de NASA para diseño y analisis de mision. Calcula orbitas, eclipses, ground station passes, ventanas de comunicacion, delta-v, tiempo de vida de bateria.

Como usarlo en tu proyecto

Una persona del equipo de operaciones deberia tener GMAT configurado con la orbita real del INTISAT. Asi se predicen los pases para downlink, los eclipses (en SAFE_MODE) y se planifica el schedule de scans.

4.3. 10. NASA Trick

- URL: <https://github.com/nasa/trick>
- Tipo: **Framework de simulacion**
- Prioridad: ★ (futuro)
- Tiempo: 1 semana

Que es: framework general de simulacion. Usado para crear simuladores de sistemas (no solo spacecraft). NASA lo usa para entrenar operadores y para testing pre-vuelo.

5. Autonomia y vision (2 referencias)

5.1. 11. NASA PLEXIL

- URL: <https://github.com/nasa/PLEXIL>
- Tipo: **Lenguaje y motor de autonomia**
- Prioridad: ★★ (interesante para tu IA)
- Tiempo: 1 semana

Que es: Plan Execution Interchange Language. Define planes ejecutables con condiciones, retries, recovery. Usado en rovers de Mars y en autonomous fault recovery.

Como usarlo en tu proyecto

Si en algun momento quieren que el payload INTISAT **decida solo** cuando hacer un scan, que mascara usar, que hacer si falla la primera, etc., PLEXIL es la herramienta. No la van a usar en la primera mision pero vale saber que existe.

5.2. 12. JPL Open Source Rover

- URL: <https://github.com/nasa-jpl/open-source-rover>
- Tipo: **Rover educativo**
- Prioridad: ★ (recreativo / didactico)
- Tiempo: 1 dia para entender

Que es: rover construible en casa con piezas COTS, basado en Curiosity y Perseverance. Documenta arquitectura mecanica + software de movilidad + vision.

Como usarlo en tu proyecto

No directamente aplicable, pero **excelente para inspiracion**. El software de control del rover (en GitHub) usa patrones similares a tu daemon: comandos, telemetria, vision.

6. ESA y la IA en orbita (3 referencias)

6.1. 13. ESA Phi-Lab

- URL: <https://github.com/ESA-PhiLab>
- Tipo: **Repositorio del laboratorio de IA de ESA**
- Prioridad: ★★★ (top — directamente alineado)
- Tiempo: 3–5 dias

Que es: la division de ESA dedicada a IA en orbita. Repos con codigo de PhiSat-1, PhiSat-2, datasets, herramientas de inferencia para Movidius Myriad 2, frameworks de evaluacion.

Repos destacados:

- `ESA-PhiLab/PhiSat-1_Coastline` — modelo de deteccion de costas
- `ESA-PhiLab/Eyes-on-the-Earth` — IA para EO en general
- `ESA-PhiLab/iris` — pipeline de inferencia on-board

Como usarlo en tu proyecto

Esto es lo mas cercano a lo que estan haciendo. Leer los papers de PhiSat-1 + el repo de `iris` les muestra:

- Como deployar un modelo entrenado a hardware embebido en orbita.
- Que arquitecturas de red funcionan en Movidius / Pi.
- Como filtrar imagenes utiles vs no utiles en tiempo real.

Lectura prioritaria del equipo.

6.2. 14. ESA Operations

- URL: <https://github.com/esa>
- Tipo: **Repositorios oficiales de ESA**
- Prioridad: ★★
- Tiempo: navegar 1 dia

Que es: repositorio principal de ESA con varios subgrupos: ECSS parsers, OPS-SAT experiments, MOOC materials, ESA herramientas.

6.3. 15. ESA OPS-SAT experiments

- URL: <https://nmf.esa.int/> y <https://github.com/esa/ops-sat-tutorials>
- Tipo: **Plataforma + tutoriales de IA en órbita**
- Prioridad: ★★★
- Tiempo: 2–3 días

Que es: OPS-SAT es el satélite de ESA para experimentar con IA y software experimental in-orbit. Sus tutoriales muestran cómo subir un experimento al satélite, correrlo, recibir resultados.

Como usarlo en tu proyecto

Inspiración directa: si en futuras misiones del INTISAT quieren correr experimentos cambiables, este es el modelo. Y si tu payload se postula a OPS-SAT VOLT como guest experiment, conocer esta plataforma es crítico.

7. CubeSats academicos open-source (3 referencias)

7.1. 16. OreSat (Portland State University)

- URL: <https://github.com/oresat>
- Tipo: CubeSat 100 % open source
- Prioridad: ★★★ (top)
- Tiempo: 1 semana para profundizar

Que es: el CubeSat mas open de la historia. Hardware, firmware, software, ground station, todo open. Diseñado por Portland State Aerospace Society (PSAS). Han volado misiones reales.

Componentes destacados:

- `oresat-c3` — Command, Control, Communications software.
- `oresat-cans` — comunicacion CAN entre subsistemas.
- `oresat-mainboard` — el carrier board del OBC.
- `oresat-firmware` — firmware multi-subsistema.

Como usarlo en tu proyecto

Es el ejemplo mas cercano a tu situacion: CubeSat academico, RPi y MCUs, comunicacion por CAN, ground station propia. Antes de inventar nada en el INTISAT, mirar como lo hicieron en OreSat.

7.2. 17. UPSat (Libre Space Foundation)

- URL: <https://gitlab.com/librespacefoundation/upsat>
- Tipo: CubeSat open-source europeo
- Prioridad: ★★
- Tiempo: 4 dias

Que es: primer CubeSat completamente open-source en volar. Lanzado en 2017 desde la ISS. Toda la documentacion es publica: requirements, diseño, codigo, tests.

Como usarlo en tu proyecto

Util para ver como se documenta un CubeSat para revisores. Si en algun momento INTISAT pasa por una review formal (Critical Design Review, Flight Readiness Review), tener UPSat de referencia ayuda mucho.

7.3. 18. AAUSAT-3 / -4 (Aalborg University)

- URL: <http://www.space.aau.dk/>
- Tipo: **Documentacion academica**
- Prioridad: ★★
- Tiempo: 2 dias

Que es: serie de CubeSats academicos de la Universidad de Aalborg (Dinamarca). Documentacion publica con modos operativos detallados, FDIR, state machines.

Como usarlo en tu proyecto

La **descripcion de modos de operacion** de AAUSAT es probablemente la mejor del mundo academico. Si vas a diseñar el diagrama de estados de mision del INTISAT, este es el template.

8. Estándares y handbooks (2 referencias)

8.1. 19. CalPoly CubeSat Design Specification

- URL: <https://www.cubesat.org/cds-announcement>
- Tipo: **Estandar mecanico/electrico**
- Prioridad: ★★★
- Tiempo: 4 horas

Que es: el estandar de facto de CubeSats. Define dimensiones, masa, conector PC-104, requisitos de testing pre-vuelo. **Lectura obligatoria** para cualquier CubeSat.

8.2. 20. NASA SE Handbook (Systems Engineering)

- URL: <https://www.nasa.gov/seh/>
- Tipo: **Manual de ingenieria de sistemas**
- Prioridad: ★★
- Tiempo: 1 semana lectura selectiva

Que es: manual oficial de NASA. Define el proceso completo: requirements, V&V, reviews, modos operativos, FDIR, configuration management.

Como usarlo en tu proyecto

No leer entero. Capítulos clave para CubeSat:

- Cap. 4 — Project Phases.
- Cap. 6 — Operational Concept Document.
- Cap. 9 — Verification & Validation.

9. Plan de exploracion para el equipo de 4 personas

Te propongo dividir el trabajo asi:

Cuadro 9.1: Asignacion de exploracion — 2 semanas.

Persona	Repos asignados	Lo que tiene que entender	Output esperado
A	OpenMCT (#3) + ESA Phi-Lab (#13)	Dashboard mission control + IA en orbita	Prototipo de UI conectado al daemon
B	cFS sample_app (#5) + LC (#6) + F'(#2)	Arquitectura pub/sub + FDIR	Doc 5 paginas con conceptos aplicables
C	OreSat (#16) + AAUSAT (#18)	CubeSat open completo + state machines	Inventario de subsistemas no documentados en INTISAT
D	42 simulator (#8) + GMAT (#9) + UPSat (#17)	Simulacion + planeamiento + docs formales	Borrador de orbita INTISAT con eclipses y pases

9.1. Cronograma

- **Dias 1–2:** cada persona instala las herramientas y corre los tutoriales basicos.
- **Dias 3–7:** profundizar en el repo asignado, tomar notas.
- **Dia 8:** reunion intermedia, brief de 10 min cada uno.
- **Dias 9–13:** profundizar en lo que se identifico como prioritario.
- **Dia 14:** reunion final — decision sobre que 3 patrones se adoptan en INTISAT.

9.2. Output del equipo al final de las 2 semanas

1. Decision sobre arquitectura: ¿adoptamos F'? ¿Mantenemos daemon Python con conceptos cFS? ¿Hibrido?
2. Prototipo de OpenMCT funcionando con telemetria del payload.
3. Borrador de modos de operacion de mision a nivel sistema.
4. Plan de orbita INTISAT con eclipses y pases.
5. Lista de patrones identificados en OreSat / AAUSAT que aplican.

10. Lectura priorizada (top 5 si solo tenes tiempo limitado)

Si el equipo no puede dedicar 2 semanas, en orden:

1. **OpenMCT tutorial** (#3) — 2 horas, salen ganando un dashboard.
2. **cFS sample_app README** (#5) — 4 horas, entiendes cFS sin perderte.
3. **F'Tutorials** (#2) — 1 dia, alternativa moderna a cFS.
4. **ESA Phi-Lab iris** (#13) — caso real de IA en orbita.
5. **OreSat C3** (#16) — ver un CubeSat open completo.

11. Recursos extra que no son repositorios pero suman

11.1. NASA Software Catalog

<https://software.nasa.gov>

Contiene código NASA NO público en GitHub. Se descarga gratis con registro. Hay herramientas valiosas (análisis térmico, control, simulación) que no están en GitHub.

11.2. Small Satellite Conference USU — Open Access Papers

<https://digitalcommons.usu.edu/smallsat/>

Todas las papers de las conferencias SmallSat de los últimos 30 años, gratis. El recurso #1 académico de CubeSats. Buscar por keyword: *state machine*, *FDIR*, *operational modes*, *onboard AI*.

11.3. ESA SAVOIR architecture

<https://savoir.estec.esa.int/>

Reference architecture de ESA para spacecraft on-board software. Documentación pública del esquema "cómo se hace un OBC en Europa".

11.4. NASA cFS resource center

<https://cfs.gsfc.nasa.gov/>

Documentación central de cFS, tutoriales, training material en video.

12. Cierre

Estas 20 referencias mas los 4 recursos extras te dan el universo de lo que existe abierto en el ecosistema CubeSat. **No es para leerlas todas**, es para que el equipo elija las 5 mas relevantes y las explore con rigor.

La leccion central: NASA, ESA, y los proyectos academicos abiertos (OreSat, UPSat, AAUSAT) llevan 30 años resolviendo los problemas de CubeSat. Antes de inventar nada en el INTISAT, vale la pena ver como ya se resolvio antes y adaptar.

Lo que NO podes copiar es la operacion: tu payload de microscopia biologica con OLED programable y mascarar intercambiables es **nuevo**. Eso es donde el equipo aporta valor diferencial. Pero la **arquitectura de software, comunicacion, FDIR y operaciones** ya esta resuelta y documentada. Hay que aprovecharla.